Quorum — Software Development Plan (ECE 49595 SD I)

Course: ECE 49595 Senior Design I, Fall 2026

Project: Quorum (AI Plan Your Trip)

Team: Team #11

Members: Aakarsh Gupta, Dylan Patel, Roshan Patel, Zachary Krause

1. Project Overview

We are building Quorum, a group trip-planning web app that uses each member's private dates, budget, airport, and preferences to make a group plan. We will use a solver to find a plan that works for everyone, show flight and hotel options, keep a shared itinerary, answer trip questions through a chat agent, and track shared expenses. Our Semester 1 goal is a working prototype for a small group, with bookings handled through links to travel providers.

2. Scope

2.1 In Scope

SYS-01 — User Constraint Intake

Description / Justification: We will let each member privately submit dates, budget, departure airport and preferences. The solver needs structured input from everyone, and private budgets are the product's core advantage over a group chat.

SYS-02 — Group Plan Solver

Description / Justification: We will combine all members' constraints into one negotiated plan (dates, destination logistics, budget split). This automates the negotiation groups now do by hand.

SYS-03 — Flight & Lodging Search

Description / Justification: We will return flight and lodging options that fit the negotiated plan, filtered by each member's airport and budget. A plan is only actionable with real options attached.

SYS-04 — Shared Itinerary

Description / Justification: We will keep one synced itinerary of confirmed flights, lodging and activities for every member. It is the group's single source of truth.

SYS-05 — Grounded Chat Agent

Description / Justification: We will answer trip questions using only the group's real data (constraints, itinerary, bookings). It gives fast answers without navigating the UI.

SYS-06 — Expense Ledger & Settlement

Description / Justification: We will track shared expenses and compute the minimum set of payments to settle balances. It removes manual cost-splitting after the trip.

SYS-07 — Authentication

Description / Justification: We will support secure sign-up and login tied to each user's constraints, bookings and expenses. Privacy (SYS-01) and attribution (SYS-04, SYS-06) depend on it.

2.2 Under Evaluation

SYS-08 — In-app booking

Open issue: Real transactions need live (non-sandbox) partner API access and payment handling; deep links may be enough for a prototype.

How we will decide: We will apply for Duffel live access now. We will decide at the midterm design review: in scope only if approved and SYS-03 is complete.

SYS-09 — Offline itinerary access

Open issue: Caching and reconciliation add complexity to SYS-04 and risk inconsistent state.

How we will decide: We will spike a cached read-only view for one sprint. We will keep it only if offline tests show no duplicate entries.

SYS-10 — Calendar export (ICS) and share link

Open issue: Useful to members, but depends on a stable fi11nalized-itinerary state.

How we will decide: We will include if FR-401 and FR-402 pass testing before the midterm demo; otherwise do this in Semester 2.

SYS-11 — Agent-proposed itinerary changes

Open issue: Letting the chat agent edit plans risks unintended changes and needs a confirmation flow.

How we will decide: We will build read-only agent first (FR-501, FR-502 and FR-503). Functionality of the agent response to trip edit requests and see if its viable.

SYS-12 — Google sign-in (SSO)

Open issue: Easier onboarding, but adds OAuth flow and a second identity path.

How we will decide: We will evaluate after email/password auth (FR-701 and FR-702) passes its security checklist.

2.3 Out of Scope

OUT-01 — Real-money settlement between users

Justification: We are leaving this out because the ledger computes who owes whom but does not move money. Payment processing brings security issues that are past a semester prototype.

OUT-02 — Multiple concurrent trips per group

Justification: We are leaving this out because one trip per group lets the solver, itinerary and ledger be tested end to end within the semester.

OUT-03 — Native mobile apps

Justification: We are leaving this out because a responsive web app covers the demo; separate iOS/Android builds would also add more work for a small team.

OUT-04 — Ground transport and car rental
Justification: Another provider integration would add work beyond our initial flight and hotel searches.

OUT-05 — Visa, passport and travel-document checks
Justification: We are leaving this out because requirements vary by nationality and change often; giving wrong guidance carries real risk to users.

2.4 Assumptions
We assume users have stable internet access while planning.

We expect groups to be small, roughly 2 to 10 members, and each member has their own account.

We assume members enter truthful constraints; the system does not verify budgets or availability.

We assume third-party travel APIs return reasonably current prices; final prices are confirmed on the provider's site via deep link.

2.5 Constraints
API access: Amadeus closed its self-service developer portal, and Skyscanner and Kiwi.com are partner-gated, limiting us to self-serve providers such as Duffel (flights) and Booking.com via RapidAPI (hotels).

Rate limits and sandbox data: Free tiers cap request volume, and Duffel's test mode returns sandbox, not live, fares.

Time: One semester for a working prototype, with a team of 4 students.

Privacy: Personal data (budgets, travel dates) must be stored and handled securely.

3. Software Requirements

3.1 Functional Requirements

FR-101 — Constraint Submission
SRID: SYS-01

Statement: When a member opens a trip they have joined, the system shall present a form capturing one or more available date ranges, a maximum budget in USD, a departure airport (IATA code) and up to 5 preference tags.

Rationale: The solver needs complete, structured input from every member.

Test Method: We will submit the form for 5 test users; verify the stored record matches every field in 5 of 5 cases.

Supporting Context: IATA code = 3-letter airport code (e.g., IND). Tags come from a fixed list (beach, city, outdoors, nightlife, food, culture).

Priority: Must Have

FR-102 — Constraint Validation

SRID: SYS-01

Statement: If a member submits a date range that ends before it starts, a budget of $0 or less, or an unknown IATA code, then the system shall reject the submission and display a field-level error message.

Rationale: Bad input produces wrong or impossible plans.

Test Method: We will submit 15 invalid inputs (5 per error type); all 15 rejected with the correct message and 0 records stored.

Supporting Context: IATA codes checked against a bundled airport list.

Priority: Must Have

FR-103 — Constraint Privacy

SRID: SYS-01

Statement: The system shall make a member's budget and date constraints visible only to that member and to the solver service.

Rationale: Private budgets are the product's core value.

Test Method: We will try to access another member's constraints 20 times through the UI and API and check that every request is denied.

Supporting Context: Solver service runs server-side; no client receives raw constraints of others.

Priority: Must Have

FR-201 — Plan Generation

SRID: SYS-02

Statement: When all members have submitted constraints, or the creator starts the solver, the system shall produce a group plan with travel dates, a per-person budget cap and a destination that satisfies every hard constraint, if a feasible plan exists.

Rationale: This reduces the manual work of comparing everyone's constraints

Test Method: We will run 20 synthetic groups with known feasible solutions; the plan satisfies all hard constraints in 20 of 20.

Supporting Context: Hard constraints = dates and budget; preferences are soft and scored.

Priority: Must Have

FR-202 — Infeasibility Report

SRID: SYS-02

Statement: If no plan satisfies all hard constraints, then the system shall name the conflicting constraint type (dates or budget) and suggest at least one compromization, without revealing any member's private values.

Rationale: We need this because groups need a path forward, and privacy must hold.

Test Method: We will run 10 infeasible groups; each report names the conflict, the suggested relaxation makes it feasible when applied, and a string search finds no private values.

Supporting Context: Example compromization: “Extending the window by 2 days finds a plan.”

Priority: Must Have

FR-203 — Plan Voting

SRID: SYS-02

Statement: When a plan is proposed, the system shall let each member accept or reject it and shall mark the plan confirmed once a majority accepts.

Rationale: Voting lets members approve the plan before booking

Test Method: We will test a four person group: three votes should confirm a plan, while two or less should not. We will repeat this with three group sizes.

Supporting Context: Majority = more than half of members.

Priority: Should Have

FR-301 — Flight Search

SRID: SYS-03

Statement: When a plan is confirmed, the system shall query the flight provider for round trip options from each member’s departure airport on the plan dates and show up to 10 lowest-priced options within that member’s budget cap.

Rationale: Members need flight options that match their departure airport and trip dates

Test Method: We will run 10 sandbox searches; every result matches the member’s airport and dates and is at or below the cap.

Supporting Context: Provider: Duffel API (sandbox during development).

Priority: Must Have

FR-302 — Lodging Search

SRID: SYS-03

Statement: When a plan is confirmed, the system shall return lodging options at the destination for the plan dates with capacity of at least the group size and a total price within the group lodging budget.

Rationale: The group usually shares lodging.

Test Method: We will run 10 searches across group sizes 2 to 10; every result meets capacity, dates and budget.

Supporting Context: Provider: Booking.com API via RapidAPI.

Priority: Must Have

FR-303 — Booking Deep Link

SRID: SYS-03

Statement: When a member selects a flight or lodging option, the system shall open the provider’s booking page for that exact option and dates.

Rationale: We need this because booking happens off-platform this semester.

Test Method: We will open 20 booking links and check that each opens the selected option and dates.

Supporting Context: Prices may change between search and checkout.

Priority: Must Have

FR-401 — Consolidated Itinerary View

SRID: SYS-04

Statement: When a group member opens the itinerary view, the system shall display all confirmed flights, lodging and activities for the trip in chronological order.

Rationale: Members need one authoritative view of the plan.

Test Method: We will seed 2 flights, 1 lodging, 3 activities; verify all 6 render in order for 3 accounts across 5 runs.

Supporting Context: “Confirmed” = locked into the group plan.

Priority: Must Have

FR-402 — Real-Time Itinerary Sync

SRID: SYS-04

Statement: When a confirmed booking changes, the system shall update the itinerary for all other members within 5 seconds.

Rationale: Stale plans cause missed flights or changes.

Test Method: We will change a booking from one session; time arrival on 2 others; pass if < 5 s in 9 of 10 trials.

Supporting Context: Requires an active connection.

Priority: Must Have

FR-405 — Conflict Flagging

SRID: SYS-04

Statement: If two itinerary items overlap in time for the same member, then the system shall flag both items visually.

Rationale: Conflict warnings help members avoid overlapping activities

Test Method: We will create overlapping and non-overlapping pairs; flag appears only on overlaps and clears on removal.

Supporting Context: Overlap = any intersecting time window.

Priority: Should Have

FR-501 — Grounded Answers

SRID: SYS-05

Statement: When a member asks the chat agent a question about the trip, the agent shall answer using only that trip’s plan, itinerary and ledger data and shall reference the items it used.

Rationale: We need this because answers must be trustworthy and checkable.

Test Method: We will ask 50 trip questions with known answers. At least 45 should be correct, and all 50 should cite trip items.

Supporting Context: Trip data is retrieved and passed to the model as context.

Priority: Must Have

FR-502 — Refuse When Unknown

SRID: SYS-05

Statement: If the trip data does not contain the answer, then the agent shall say it does not have that information instead of guessing.

Rationale: Made-up flight times or prices could strand a traveler.

Test Method: We will ask 20 questions that our trip data cannot answer. At least 19 should be declined, with no made-up booking details.

Supporting Context: Fabricated = any time, price or code not in trip data.

Priority: Must Have

FR-503 — No Private Data Leakage

SRID: SYS-05

Statement: The agent shall not reveal any member's private constraints to another member.

Rationale: Private constraints must remain protected when members use the chatbot

Test Method: We will try 30 prompts asking for another member's private information and verify none of the answers reveals it.

Supporting Context: Agent context excludes other members' raw constraints.

Priority: Must Have

FR-601 — Expense Entry

SRID: SYS-06

Statement: When a member records an expense, the system shall store its amount, payer, participating members and split method (equal or custom amounts).

Rationale: We need this because settlement depends on complete records.

Test Method: We will enter 10 expenses of each split type; stored records match input in all 20.

Supporting Context: Custom amounts must sum to the expense total.

Priority: Must Have

FR-602 — Balance Calculation

SRID: SYS-06

Statement: The system shall compute each member's net balance so that all balances in a trip sum to exactly $0.00.

Rationale: Errors here cause issues with who is paying more or less in the group.

Test Method: We will run 100 randomized ledgers; balance sum equals 0 cents in 100 of 100.

Supporting Context: Computed in integer cents (NFR-601).

Priority: Must Have

FR-603 — Settlement Plan

SRID: SYS-06

Statement: When a member requests settlement, the system shall produce a list of at most n−1 transfers (n = members with nonzero balance) that brings every balance to $0.00.

Rationale: We need this because fewer transfers means less friction after the trip.

Test Method: We will run 100 randomized ledgers; residual 0 and transfers less than or equal to n−1 in all; for groups less than or equal to 6, compare count to a brute-force minimum and report the gap.

Supporting Context: Exact minimum is NP-hard in general; a greedy match guarantees less than or equal to n−1.

Priority: Must Have

FR-701 — Account Sign-Up

SRID: SYS-07

Statement: When a new user submits an email and a password of at least 8 characters that is not on a breached-password list, the system shall create an account and send a verification email.

Rationale: Every feature depends on identity.

Test Method: We will test 10 valid sign-ups and 10 invalid passwords, checking that valid accounts receive email verification and invalid passwords are rejected.

Supporting Context: Breach check against a known-compromised list.

Priority: Must Have

FR-702 — Login

SRID: SYS-07

Statement: When a user submits valid credentials for a verified account, the system shall start an authenticated session; if credentials are invalid, then the system shall show one generic error that does not reveal which field was wrong.

Rationale: We need this because prevents account enumeration.

Test Method: We will test 10 valid logins and check that incorrect email and password attempts produce the same generic error.

Supporting Context: Session = server-issued token in an HttpOnly, Secure cookie.

Priority: Must Have

3.2 Non-Functional Requirements

NFR-101 — Data Protection (Security)

SRID: SYS-01

Statement: The system shall encrypt all constraint and account data in transit with industry standards.

Rationale: We need this because budgets and travel dates are personal data.

Test Method: We will scan the deployed site with the industry standard security and confirm that stored data is encrypted.

Supporting Context: Managed database encryption at rest is acceptable.

Priority: Must Have

NFR-201 — Solver Response Time (Performance)

SRID: SYS-02

Statement: When the solver is started for a group of up to 10 members, the system shall return a plan or an infeasibility report within 10 seconds.

Rationale: Long waits break the planning flow.

Test Method: We will run 50 synthetic groups of 2 to 10 members; 95th-percentile time < 10 s.

Supporting Context: Measured server-side, excluding provider searches.

Priority: Must Have

NFR-301 — Search Response Time (Performance)

SRID: SYS-03

Statement: When a member requests flight or lodging results, the system shall display the first results within 10 seconds in 95% of searches.

Rationale: Members abandon slow searches.

Test Method: We will run 40 searches across four routes and check that 95% show results within eight seconds.

Supporting Context: Includes provider API latency.

Priority: Should Have

NFR-302 — Provider Failure Handling (Reliability)

SRID: SYS-03

Statement: If a provider API returns an error or does not respond within 10 seconds, then the system shall show a retry option and any cached results less than 24 hours old.

Rationale: We need this because free tier APIs fail and throttle.

Test Method: We will inject 500 errors and timeouts with a mock provider; UI shows retry and cached results in 10 of 10 cases, no crash.

Supporting Context: Cache keyed by route and dates.

Priority: Should Have

NFR-401 — Sync Latency Under Load (Performance)

SRID: SYS-04

Statement: The system shall propagate itinerary updates to all members within 5 seconds for groups of up to 10.

Rationale: The size of the group should not increase the sync time.

Test Method: We will run 10 synthetic clients, 20 update events; 95th-percentile latency < 5 s.

Supporting Context: Regular high speed internet/mobile connection using 4G or faster

Priority: Must Have

NFR-403 — Itinerary Consistency (Reliability)

SRID: SYS-04

Statement: When sync completes, the system shall present an identical itinerary to every member.

Rationale: We need this because divergent views defeat a shared plan.

Test Method: We will hash the itinerary payload on all clients after 10 sync events; 100% match.

Supporting Context: Post-sync steady state only.

Priority: Must Have

NFR-501 — Agent Response Time (Performance)

SRID: SYS-05

Statement: When a member sends a chat message, the system shall begin showing the agent's reply within 6 seconds in 95% of messages.

Rationale: We need this because chat must feel faster than navigating the UI.

Test Method: We will run 50 benchmark questions; 95th-percentile time to first token < 6 s.

Supporting Context: Streaming responses.

Priority: Should Have

NFR-601 — Monetary Precision (Reliability)

SRID: SYS-06

Statement: The system shall store and compute all monetary amounts as integer cents.

Rationale: Floating-point rounding creates balances that do not sum to zero.

Test Method: We will review the ledger code and test 1,000 random expense splits, checking that balances always total zero cents.

Supporting Context: Remainder cents assigned deterministically.

Priority: Must Have

NFR-702 — Access Control (Security)

SRID: SYS-07

Statement: The system shall require an authenticated session for every API endpoint except sign-up, login, password reset and read-only share links.

Rationale: One open route can expose private data.

Test Method: We will automatically call every API route without signing in and check that only the allowed public endpoints work.

Supporting Context: Route list generated from the server router.

Priority: Must Have

4. Deliverables

D-1 — Deployed web prototype

We will deliver a hosted web app for one group trip: account sign-up/login, private constraints, group plan and voting, flight/lodging search with booking links, synced itinerary, read-only chat agent, and expense settlement.

Requirements satisfied: FR-101, FR-102, FR-103, FR-201, FR-202, FR-203, FR-301, FR-302, FR-303, FR-401, FR-402, FR-405, FR-501, FR-502, FR-503, FR-601, FR-602, FR-603, FR-701, FR-702

D-2 — Source repository

We will provide a GitHub repository with setup instructions, project code, and automated tests.

Requirements satisfied: NFR-702, NFR-403

D-3 — Verification test report

We will report pass/fail results for all 20 FRs and 10 NFRs, including search, sync, privacy, and performance tests.

Requirements satisfied: FR-101, FR-102, FR-103, FR-201, FR-202, FR-203, FR-301, FR-302, FR-303, FR-401, FR-402, FR-405, FR-501, FR-502, FR-503, FR-601, FR-602, FR-603, FR-701, FR-702, NFR-101, NFR-201, NFR-301, NFR-302, NFR-401, NFR-403, NFR-501, NFR-601, NFR-702

D-4 — Security review

We will document authentication, permission, and private-constraint tests with API access checks.

Requirements satisfied: FR-103, FR-503, FR-701, FR-702, NFR-101, NFR-702

D-5 — Chat agent evaluation

We will evaluate the agent using answerable, unanswerable, and privacy/adversarial prompt tests with response-time results.

Requirements satisfied: FR-501, FR-502, FR-503, NFR-501

D-6 — Solver and ledger benchmark

We will benchmark feasible/infeasible group tests, solver timing, balance accuracy, and settlement checks.

Requirements satisfied: FR-201, FR-202, FR-602, FR-603, NFR-201, NFR-601

D-7 — Documentation package

We will provide a user guide, architecture overview, API setup notes, and deployment instructions.

Requirements satisfied: FR-301, FR-302, FR-303, NFR-302

D-8 — Final demo video

We will record a five minute walkthrough from creating an account through group planning and final expense settlement.

Requirements satisfied: FR-101, FR-102, FR-103, FR-201, FR-202, FR-203, FR-301, FR-302, FR-303, FR-401, FR-402, FR-405, FR-501, FR-502, FR-503, FR-601, FR-602, FR-603, FR-701, FR-702

5. Standards

OWASP ASVS 5.0 (https://owasp.org/www-project-application-security-verification-standard/)

Why it applies: We store accounts, private budgets, and travel dates.

Impact on design and code: We use this standard for password storage, sessions, input validation, and API access control.

How we ensure compliance: We will check Level 1 security requirements each sprint and document results in D-4.

NIST SP 800-63B-4 (https://pages.nist.gov/800-63-4/sp800-63b.html)

Why it applies: We need security guidance for passwords and login in SYS-07.

Impact on design and code: We use its guidance for password rules, breached-password checks, and login protection.

How we ensure compliance: We will compare our authentication features with NIST guidance and test them.

WCAG 2.2 (https://www.w3.org/TR/WCAG22/)

Why it applies: We want every member to be able to submit constraints and read the itinerary.

Impact on design and code: We will use clear labels, keyboard navigation, visible focus, and readable contrast.

How we ensure compliance: We will run automated checks and a manual screen-reader test for D-5.

ISO/IEC 25010:2023 (https://www.iso.org/standard/78176.html)

Why it applies: We use this standard to define software quality goals.

Impact on design and code: We will measure performance, reliability, security, and maintainability in our NFRs.

How we ensure compliance: We will connect each NFR to a test.

RFC 9700, OAuth 2.0 Security BCP (https://www.rfc-editor.org/rfc/rfc9700)

Why it applies: We would need it if we add Google sign-in.

Impact on design and code: We would use PKCE and secure redirect handling for OAuth.

How we ensure compliance: We will review this standard if SSO becomes part of our scope.

6. References

OWASP Foundation, “Application Security Verification Standard 5.0,” 2025. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

National Institute of Standards and Technology, “Digital Identity Guidelines: Authentication and Authenticator Management,” NIST SP 800-63B-4, 2025. [Online]. Available: https://pages.nist.gov/800-63-4/sp800-63b.html

World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, 2023. [Online]. Available: https://www.w3.org/TR/WCAG22/

ISO/IEC 25010:2023, “Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model,” ISO, 2023. [Online]. Available: https://www.iso.org/standard/78176.html

T. Lodderstedt, J. Bradley, A. Labunets and D. Fett, “Best Current Practice for OAuth 2.0 Security,” RFC 9700, IETF, Jan. 2025. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9700

Duffel, “Duffel API Documentation,” 2026. [Online]. Available: https://duffel.com/docs

Thunderbit, “10 Best Flight APIs in 2026: Free Tiers and Real Pricing,” Aug. 2026. [Online]. Available: https://thunderbit.com/blog/best-flight-api-with-free-tiers

Ignav, “Best Flight APIs for Developers,” Aug. 2026. [Online]. Available: https://ignav.com/docs/comparisons

Expedia Group, “Rapid API Explorer,” 2026. [Online]. Available: https://developers.expediagroup.com/docs/products/rapid/api/explorer

7. GenAI Statement

We used generative AI to help shorten the software requirements and update the deliverable mappings. We got our standards and sources from ChatGPT. The team reviewed the requirements, test methods, and project scope and will verify them before submission.

8. Individual Contributions

The team will update its individual contribution table with the actual work completed for this revision.

Aakarsh Gupta: Shared itinerary, authentication, security review

Dylan Patel: Constraint intake and group solver

Roshan Patel: Flight/lodging search and API integration

Zachary Krause: Chat agent and expense ledger